

*******JAVA*******

Java intro

- jshell - you can experiment and learn by writing statements in java without class and main method invented from java9
- java is platform independent where as JDK,JVM,JRE are platform dependent
- jvm only accepts byte code
- javac ie java compiler converts our code into byte code
- when we run the application jvm looks for the signature (main method) and identifies it as the start of the execution
- javac FileName.java (To compile java file)
- java Hello (To execute java file) JVM -java virtual machine

OS lies on hardware

JRE Lies on OS

JRE- Java Runtime Environment - JVM along with libraries

The JDK (Java Development Kit) is a software development kit that develops applications in Java,
JDK contains JRE and JRE contains JVM

Just to run java application we don't need JDK ,that's why we says java is wora - write once run anywhere provided machine had JRE

JDK = Development Tools + JRE (Java Runtime Environment)

JRE = Libraries for running the application + JVM (Java Virtual Machine)

JVM = Only the runtime environment that helps in executing the Java bytecode.

Naming Convention :

package- all small letters

{

The prefix of a unique package name is always written in all-lowercase ASCII letters and should be one of the top-level domain names, currently com, edu, gov, mil, net, org, or one of the English two-letter codes identifying countries as specified in ISO Standard 3166, 1981.

Subsequent components of the package name vary according to an organization's own internal naming conventions. Such conventions might specify that certain directory name components be division, department, project, machine, or login names.

}

Class (noun) - 1st letter Capital

Interface - 1st letter Capital

method(verb) - camel case

variables - camel case

constants - All Capitals separated by underscores

Any way an identifier in java starting with _ and \$ don't cause any error but not preferred according to convention

Variable names are case-sensitive. A variable's name can be any legal identifier — an unlimited-length sequence of Unicode letters and digits, beginning with a letter, the dollar sign "\$", or the underscore character "_". The convention, however, is to always begin your variable names with a letter, not "\$" or "_". Additionally, the dollar sign character, by convention, is never used at all. You may find some situations where auto-generated names will contain the dollar sign, but your variable names should always avoid using it. A similar convention exists for the underscore character; while it's technically legal to begin your variable's name with "_", this practice is discouraged. White space is not permitted.

Subsequent characters may be letters, digits, dollar signs, or underscore characters.

Conventions (and common sense) apply to this rule as well. When choosing a name for your variables, use full words instead of cryptic abbreviations. Doing so will make your code easier to read and understand. In many cases it will also make your code self-documenting; fields named cadence, speed, and gear, for example, are much more intuitive than abbreviated versions, such as s, c, and g. Also keep in mind that the name you choose must not be a keyword or reserved word.

If the name you choose consists of only one word, spell that word in all lowercase letters. If it consists of more than one word, capitalize the first letter of each subsequent word. The names gearRatio and currentGear are prime examples of this convention. If your variable stores a constant value, such as static final int NUM_GEAR = 6, the convention changes slightly, capitalizing every letter and separating subsequent words with the underscore character. By convention, the underscore character is never used elsewhere.

`System.out.print --> prints`

`System.out.println --> prints and goes to nextline`

`println` belongs to `PrintStream` class and if we want to use `println` we need `PrintStream` object (`out`) which is a static object of `System` class

`System.in.read()` gives ascii value (int format)

- `java.lang.*` would be automatically imported but having a practice of writing import at start is good
- In java everything should be written inside classes
- Starting point of java program is main method
- JVM is an interpreter it converts byte code into machine code
- If we want anything from class accessible outside, it should be public
- When we executed a java program JVM calls main method JVM in order to see main method it should be public
- In order to use anything from class just with class name without creating object we have to use static. JVM has to call main method without creating an object hence we keep it static
- `println` is a method used to print present inside static object out present inside class `System`
- method should have bracket
- It's better to have class name and file name same

or else it'd be

`javac First.java`

`java Second`

- when class is created as public then definitely that class name and file name should be same
- even if we didn't write command line arguments (`String args[]`) in main method it will be compiled but not executed

`javac First.java`

`java First Hi All`

`sys.out arg[0]=>Hi`

`sys.out arg[1]=>All`

- `util` package contains `Scanner` class
- So we have to import `java.util.Scanner`
- `Scanner` class is provided from v5 onwards
- `System.in` is associated with keyboard

Variables

The Java programming language defines the following kinds of variables:

Instance Variables (Non-Static Fields) Technically speaking, objects store their individual states in "non-static fields", that is, fields declared without the static keyword. Non-static fields are also known as instance variables because their values are unique to each instance of a class (to each object, in other words); the `currentSpeed` of one bicycle is independent from the `currentSpeed` of another.

Class Variables (Static Fields) A class variable is any field declared with the static modifier; this tells the compiler that there is exactly one copy of this variable in existence, regardless of how many times the class has been instantiated. A field defining the number of gears for a particular kind of bicycle could be marked as static since conceptually the same number of gears will apply to all instances. The code `static int numGears = 6;` would create such a static field. Additionally, the keyword `final` could be added to indicate that the number of gears will never change.

Local Variables Similar to how an object stores its state in fields, a method will often store its temporary state in local variables. The syntax for declaring a local variable is similar to declaring a field (for example, `int count = 0;`). There is no special keyword designating a variable as local; that determination comes entirely from the location in which the variable is declared — which is between the opening and closing braces of a method. As such, local variables are only visible to the methods in which they are declared; they are not accessible from the rest of the class.

and Parameters

It's not always necessary to assign a value when a field is declared. Fields that are declared but not initialized will be set to a reasonable default by the compiler. Generally speaking, this default will be zero or null, depending on the data type. Relying on such default values, however, is generally considered bad programming style.

The following chart summarizes the default values for the above data types.

Data Type	Default Value (for fields)
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
String (or any object)	null
boolean	false

Local variables are slightly different; the compiler never assigns a default value to an uninitialized local variable. If you cannot initialize your local variable where it is declared, make sure to assign it a value before you attempt to use it. Accessing an uninitialized local variable will result in a compile-time error.

Datatypes

The Java programming language is statically-typed, which means that all variables must first be declared before they can be used

It is also strongly-typed , which means a programming language in which each type of data, such as integers, characters, hexadecimals and packed decimals, is predefined as part of the programming language, and all constants or variables defined for a given program must be described with one of the data types.

Local variables are slightly different; the compiler never assigns a default value to an uninitialized local variable. If you cannot initialize your local variable where it is declared, make sure to assign it a value before you attempt to use it. Accessing an uninitialized local variable will result in a compile-time error.

Data types in Java are of different sizes and values that can be stored in the variable that is made as per convenience and circumstances to cover up all test cases. Java has two categories in which data types are segregated

Primitive Data Type: such as boolean, char, int, short, byte, long, float, and double

Non-Primitive Data Type or Object Data type: such as String, Array, etc.

Type	Description	Default	Size	Example Literals	Range of values
boolean	true or false	false	8 bits	true, false	true, false
byte	twos-complement integer	0	8 bits	(none)	-128 to 127
char	Unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\', '\n', 'ß'	characters representation of ASCII values 0 to 255
short	twos-complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos-complement integer	0	32 bits	-2,-1,0,1,2	-2,147,483,648 to 2,147,483,647
long	twos-complement integer	0	64 bits	-2L,-1L,0L,1L,2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f , -1.23e-100f , .3f ,3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d , -123456e-300d , 1e1d	upto 16 decimal digits

Literals

Values stored in variables are known as literals

Ex: int i =1 where i is variable and 1 is literal

0s in a literal can be separated by underscores to avoid confusion

Working with Binary , HexaDecimal and Octal

->To work with binary numbers - 0b101

(101 is binary) (if we print this 5 will be printed)

->To work with Hexadecimal numbers - 0x7E

(7E is Hexadecimal) (if we print this 126 will be printed)

->To work with Octal numbers - 0157

(157 is Octal) (if we print this 111 will be printed)

Parsing Numbers in Different Bases:

```
String num = "100"
```

```
int num1 = Integer.parseInt(num, 8); // decimal value 100 is being converted to Octal
```

```
int num2 = Integer.parseInt(num, 16); // decimal value 100 is being converted to Hexadecimal
```

```
int num = Integer.parseInt(num, 2); //decimal value 100 is being converted to Binary
```

Converting Between Number Systems:

```
String hexStr = "0x100"; // Hexadecimal representation
```

```
int decimalValue = Integer.decode(hexStr); // Converts hexadecimal to decimal
```

```
int decimalValue = 256; // Decimal value
```

```
String hexStr = Integer.toHexString(decimalValue); // Converts decimal to hexadecimal
```

```
String octalStr = "0100"; // Octal representation
```

```
int decimalValue = Integer.decode(octalStr); // Converts octal to decimal
```

```
int decimalValue = 64; // Decimal value
```

```
String octalStr = Integer.toOctalString(decimalValue); // Converts decimal to octal
```

```
String binaryStr = "0b100"; // Binary representation
```

```
int decimalValue = Integer.parseInt(binaryStr.substring(2), 2);
```

```
int decimalValue = 4; // Decimal value
```

```
String binaryStr = Integer.toBinaryString(decimalValue); // Converts decimal to binary
```

String

String is a sequence of characters which is immutable

Syntax : `String str = new String()` (or) `String str = " "`

The Constructor can be empty, can accept another string, a character array (whole or part), a byte array (whole or part), or a byte array with a specified charset in its constructor.

`String str = "foo"` // only object will be created in string constant pool for literal bcoz new is not used

`String str = new String("foo")` // one object in heap , another in string constant pool

String is immutable only one object will be created in string constant pool and if literals are same , references will point to that same object

StringBuffer and StringBuilder are mutable

StringBuffer is thread safe where as StringBuilder is not

Syntaxes for StringBuffer and StringBuilder :

`StringBuffer sb = new StringBuffer();`

`StringBuilder sbd = new StringBuilder();`

Both StringBuffer and StringBuilder can be instantiated with no arguments (creating an empty object with a default capacity of 16), with an initial string (initializing with the content of the provided string), or with an initial capacity (setting the specified initial capacity).

Feature	String	StringBuilder	StringBuffer
Introduction	Introduced in JDK 1.0	Introduced in JDK 1.5	Introduced in JDK 1.0
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread Safe	Not Thread Safe	Thread Safe
Memory Efficiency	High	Efficient	Less Efficient
Performance	High(No-Synchronization)	High(No-Synchronization)	Low(Due to Synchronization)
Usage	This is used when we want immutability.	This is used when Thread safety is not required.	This is used when Thread safety is required.

Type Conversion

Can we change a type of the variable - No but we can assign the value to types

Casting - Explicit Conversion

A variable of smaller data type can be stored in bigger data type but the vice versa can not be done due to chance of loss of data in order to achieve this we can type cast the data

int a = b will work (Type Conversion)

byte b = a will not work -> byte b = byte(a) (Type Casting)

In the case of type casting if datatype size is smaller than the value that has to be stored % operator will be used

ex: int a = 257;

byte b = (byte) a;

257 % 256 (which is byte size) = 1, so 1 will be stored in b in this case

Operators

1. Arithmetic Operators

+ (addition): a + b

- (subtraction): a - b

* (multiplication): a * b

/ (division): a / b

% (modulus): a % b

2. Unary Operators

Operate on a single operand.

+ (unary plus): +a

- (unary minus): -a

++ (increment): a++ or ++a

-- (decrement): a-- or --a

! (logical NOT): !a

3. Assignment Operator

Assigns a value to a variable.

`=: a = b`

`+=: a += b` (equivalent to `a = a + b`)

`-=: a -= b` (equivalent to `a = a - b`)

`*=: a *= b` (equivalent to `a = a * b`)

`/=: a /= b` (equivalent to `a = a / b`)

`%=: a %= b` (equivalent to `a = a % b`)

4. Relational Operators

Compare two values and return a boolean result.

`==` (equal to): `a == b`

`!=` (not equal to): `a != b`

`>` (greater than): `a > b`

`<` (less than): `a < b`

`>=` (greater than or equal to): `a >= b`

`<=` (less than or equal to): `a <= b`

5. Logical Operators

Combine boolean expressions.

`&&` (logical AND): `a && b`

`||` (logical OR): `a || b`

`!` (logical NOT): `!a`

6. Ternary Operator

A shorthand for if-else statements.

Syntax: `condition ? ifTrue : ifFalse`

Example: `int max = (a > b) ? a : b;`

7. Bitwise Operators

Perform operations on bits of integer values.

& (bitwise AND): $a \& b$

| (bitwise OR): $a | b$

^ (bitwise XOR): $a \wedge b$

~ (bitwise complement): $\sim a$

8. Shift Operators

Shift bits left or right.

<< (left shift): $a \ll 2$ (shifts bits of a two positions to the left)

>> (right shift): $a \gg 2$ (shifts bits of a two positions to the right)

>>> (unsigned right shift): $a \ggg 2$

9. instanceof Operator

Tests whether an object is an instance of a specific class or subclass.

Syntax: `object instanceof ClassName`

Example: `if (str instanceof String)`

Conditional Statements

if Statement: Executes a block of code if a specified condition evaluates to true.

if-else Statement: Executes one block of code if the condition is true, and another block if the condition is false.

nested-if Statement: Uses an if statement inside another if statement, allowing for more complex decision-making.

if-else-if Statement: Tests multiple conditions in sequence, executing the code block for the first true condition, or the final else block if none are true.

switch-case Statement: The **switch statement in Java** is a multi-way branch statement. In simple words, the Java switch statement executes one statement from multiple conditions.

It is an alternative to an [if-else-if](#) ladder statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression. The expression can be of type byte, short, char, int, long, enums, String, or wrapper classes (Integer, Short, Byte, Long).

Syntax for switch case: `switch (choice){ case 1:// Code for case 1 break; case 2: // Code for case 2 break; case 3: // Code for case 3 break; default: // Code for default case break; }`

Jump statements

break Statement: Exits from the nearest loop or switch statement, stopping further execution of the loop or switch block.

continue Statement: Skips the current iteration of a loop and proceeds with the next iteration, useful for bypassing specific cases in loop execution.

return Statement: Exits from a method and optionally provides a result back to the caller, ending the method's execution.

Looping Statements

while Loop: Repeats a block of code as long as the specified condition is true.

do-while Loop: Similar to the while loop but guarantees that the code block is executed at least once before checking the condition.

for Loop: Iterates a block of code a specific number of times using an initialization, condition, and increment/decrement expression.

Enhanced for Loop (for-each): Used specifically for iterating over elements in collections, arrays, or other iterable objects.

Syntax: `for(data-type variable : datastructure used)`

`{// Code to be executed}`

Additional Iteration Techniques in Java

Stream API (forEach): Introduced in Java 8, the Stream API provides powerful ways to iterate over collections with lambda expressions using `forEach()` and other operations like `map`, `filter`, and `reduce`.

Iterator and ListIterator: These interfaces provide methods (`hasNext()`, `next()`, `remove()`) to iterate over collections in a controlled manner, allowing safe modifications during traversal.

Recursion: Although not a loop, recursion allows repetitive execution by having a method call itself with a modified parameter until a base condition is met.

Streams with Collectors: Often used to iterate and collect results into different forms, e.g., lists, sets, or maps.

Parallel Streams: Allows parallel iteration over collections, which can be beneficial for performance in multi-core processors.

Arrays.parallelPrefix, Arrays.parallelSetAll: Special methods in Arrays that allow parallel operations, especially in manipulating arrays.

Reading Input :

```
BufferedReader reader = new BufferedReader(new InputStreamReader(System.in))
```

//can read from Standard i/p

reader.readLine() to read values - return String

```
Scanner sc = new Scanner(System.in)
```

//can read from Standard i/p

sc contain different methods for different type of inputs

```
DataInputStream reader = new DataInputStream(System.in);
```

//can read from Standard i/p

reader.readLine() // deprecated ,don't use this

```
Console reader = System.console() // only from console , couldn't read from files
```

//can read from only console , return null if console is not available

console.readLine() ,console.readPassword

Command line arguments // passed as arguments when program is run

Whatever resource is used, close it after the purpose of it is completed.

Closing resources is a good practice , or else resources will be leaked , if not closed others cant use

finally is a block which will be executed default irrespective of exception , finally is preferred for closing resources

try with resources - autoclosing resource by creating object in try() inside paranthesis

Methods

- First line of method is known as signature or header or heading
- Java does not support default parameter values in method definitions.
- Every method will have its own stack
- No, each method in Java does not have its own heap. The heap is a shared memory space that is used to store all objects created in the Java Virtual Machine (JVM)
- A static method can only call static method or else we need object of that method class
- The parameters we are passing are actual parameters
- The parameters taking those are formal parameters(take the copy of actual parameters)
- ****Content of the actual parameter is copied to formal parameter****
- In case of object it would be its reference (Array:Address)
- We will take array as formal parameter as - type arr_name[], actual parameter will be taken as arr_name another array wont be formed at method, formal parameter just take reference (points to same address ((actually no pointing and address in java just for understanding purpose)))
- when we pass object as parameter object, object itself wont be passed but its reference will be passed but when modification is done due to implementation of method actual object will be modified
- Object can also be return type
- Java didnt have various methods like call by value or call by reference its just parameter passing because java dont have pointer concept
- The calling method is the method that contains the actual call.
- The called method is the method being called. They are different.
- They are also called the Caller and the Callee methods.

For example:

```
int caller()
{
int x=callee();
}
int callee()
{
return 5;
}
```

Type of Modularisations:

Method level modularisation - All methods in same class

Class level modularisation - Among several Classes in a package

Package level modularisation - Among different Packages

Either inside or outside class instance needs object , static needs class name (for static inside the same class even without class name also we can access)

Access modifiers:

public

default

protected

private

public, default - other class can view

private - other class can't view

protected only child or parent or any class in the family hierarchy

We could only import class , we could not import packages

Method overloading : Methods with same name and different parameter list(either different datatypes or different no of parameters)

```
class Example {
    void display(int a) {System.out.println("Parameter: " + a);}
    void display(int a, int b) { System.out.println("Parameters: " + a + ", " + b);}
    void display(double a) {System.out.println("Parameter: " + a);}
    public static void main(String[] args) {
        Example obj = new Example();
        obj.display(5);      // Calls method with int parameter
        obj.display(5, 10);  // Calls method with two int parameters
        obj.display(5.5);    // Calls method with double parameter
    }
}
```

Arrays:

Array is a collection of similar datatype elements

```
int a[ ] = new int[size];
```

or also

```
int a[ ]={1,2,3,4,5} // this is also possible , we can only give values like this at the time of declaration
```

(If we want to give all values at a time it is only possible during declaration)

if value is not allotted it will be 0

Printing Array using different loops:

using for loop:

```
for (int i = 0; i < nums.length; i++) {  
    System.out.print(nums[i] + " ");  
}
```

using for each loop:

```
for (int num : nums) {  
    System.out.print(num + " ");  
}
```

using while loop:

```
int i = 0;  
while (i < nums.length) {  
    System.out.print(nums[i] + " ");  
    i++;  
}
```


Array features:

- The memory allocation is contiguous.
- The size of an array is fixed. Array size cannot be expand.
- Searching takes time.
- Array can store values of only same type. It can store homogeneous type value only.

Multidimensional array- An array containing arrays

`data_type array_name [] [] = new data_type [m] [n]` => multidimensional array containing m arrays and each array having n elements

ex: `int nums [] [] = new int [3] [4]`

```
nums =>
      0  1  2  3
0     5  6  7  8
1     9 10 11 12
2    13 14 15 16
```

`nums[0][0]` is 5, `nums[0][3]` is 8, `nums[2][2]` is 15

nested loops are used for multidimensional array

using for loop:

```
for(int i=0;i<3;i++){
    for(int j=0;j<4;j++){
        System.out.println(nums[i][j]+" ");
    }
    System.out.println();
}
```

using for each loop:

```

        for(int n[:nums)
        {
            for(int m:n)
            {
                System.out.println(m+" ");
            }
            System.out.println();
        }

```

using while loop:

```

int i = 0;
while (i < nums.length) {
    int j = 0;
    while (j < nums[i].length) {
        System.out.print(nums[i][j] + " ");
        j++;
    }

    System.out.println();
    i++;
}

```

Bigger method (parameters such as float)work for smaller paramters (parameters such as int)also but not viceversa

Variable arguments (varargs):

we can pass any no of arguments during passing actual parameters without mentioning at formal parameters

but type of parameters should be same here

`show(int ...x) // formal`

actual:

`show(1)`

`show(1,2)`

`show(1,2,..)`

we can even pass array for varargs

`show(new int[]{10,20,30,40})`

We cannot take multiple varargs

and varargs should be last parameter

`show(int x,int ...y)`

java also contain `System.out.printf()` similar to `printf` in c

`printf` in c is based on ellipsis(varargs concept in c and c++)

In psvm for command line args we can also use varargs

`String ...args`

*****OOPS*****

OBJECT ORIENTED PROGRAMMING(OOP)

OOPS is used to achieve advantages of some features in general engineering

OOPS Concepts include:

Object

Class

Inheritance

Polymorphism

Abstraction

Encapsulation

The object is an instance of a class. A class is like a blueprint or template that defines the properties and behavior of objects. When we create an object we are creating an instance of that class.

Class : a specification, blueprint for an object...

Object : physical presence of the class in memory...

Instance : a unique copy of the object (same structure, different data)...

In Java, the terms "object" and "instance" are often used interchangeably. However, there is a subtle difference between the two terms. An object is a more general term, while an instance is a more specific term.

An object can be any entity that has its own identity, behavior, and state. This includes objects that are not created from a class blueprint. For example, a string object is an object, but it is not created from a class blueprint.

An instance is a specific realization of a class. It is a concrete object that exists in memory. An instance has its own unique values for its data members.

- Any real world entity is an object
- object is defined based on it's properties(nouns)-data members and behaviour(verbs)-methods
- behaviour will effect the properties i.e. utilize properties may be using them or changing them
- For constructing object we need a (design (Class)-defining the design)
- One Class can have many objects
- Abstraction means hiding internal details and showing only required features
- Encapsulation provides a single body(encapsulate everything - boxing up)
- Inheritance takes the same features from back (inheriting them)with some additional features
- Object is created as new Classname()
- Reference is created in stack(method area), object is stored in heap

Encapsulation

Encapsulating into a single body

Data Hiding

It just provides a way to protect your data from the outside world. What it means is, lets say if I made my instance variable public, then anyone can change its state. But if we make our instance variable private/protected then actually we are restricting outside entities from making changes to it.

They ask for change through method and we decide it to be done or not

So properties can not be used directly only way to access them is through methods, we use them utilizing getter, setter methods

Constructors

- Constructor is a method of class which is automatically called when an object is created
- Constructors are used in Java to initialize objects
- Every class in java will have default constructor
- when we define our own constructor default constructor will be removed
- constructor name is same as class name
- constructor don't have a return type

Constructor are of two types:

1. parameterized
 2. non parameterized
-
- Constructor overloading is possible
 - Constructor chaining is the process of calling one constructor from another constructor with respect to current object.
 - One of the main use of constructor chaining is to avoid duplicate codes while having multiple constructor (by means of constructor overloading) and make code more readable.

this and super

"this" is a keyword which is used to access instance members of class corresponding to the current object

"this()" is a method which is used to call constructor of same class

"super" is a keyword used to access parent class properties

"super()" is a method which is used to called parent class constructor

Every constructor calls super() implicitly - so on default non-parameterized constructor of parent will be executed when constructor of child class is called

Array of objects:

Array of objects not directly contain objects it contains references to objects

ex: There is a class Subject

Syntax:

```
Subject subs[] = new Subject[size]
```

subs is the array that contains references

Inheritance (Specialization)

Inheritance is the process of acquiring the features of an existing class into a new class.

So acquiring the properties and methods of an existing class into a new class is inheritance.

Inheritance works with class files not java files

extends keyword is used for inheritance

Every class extends Object class implicitly in java until we extend something else

Syntax:

```
ChildClass extends ParentClass{}
```

for example, if there is a class called circle, then a class called cylinder, circle class, which is having only radius property, then a cylinder will also have height, radius and height. So we can say that cylinder can acquire all the features of a circle, plus it can have extra properties that is height. So we can write a class cylinder inheriting from circle.

class called cylinder, which is extending from circle. So everything from the circle class will be available in cylinder class, available, it will be available. And second term is whether everything is accessible or not

Types of Inheritance :

1. Single Inheritance <-> Single Parent - Single child
2. Multilevel Inheritance <-> Parent - Child - Grandchild ..
3. Hierarchical Inheritance Parent <-> Parent - n no of childs
4. Multiple Inheritance <-> Single child - n no of parents

Multiple Inheritance--> multiple inheritance in Java will not work. (ambiguity problem - let say A extends B,C and obj.x() is called x is not there in child then it goes to parent and search in general now if both B and C classes contain x() method which one to choose this is ambiguity problem) to avoid this java didn't had the feature of multiple inheritance

- Can be achieved through interfaces

5. Hybrid Inheritance <-> composition of two or more types of inheritance

Every class in java extends Object class even if we don't mention it explicitly

Overriding: Child class to provide a specific implementation of a method that is already provided by its Parent class. When a method in a subclass has the same name, the same parameters or signature, and the same return type(or sub-type) as a method in its super-class, then the method in the subclass is said to *override* the method in the super-class.

Syntax:

```
class Parent {
    void display() {
        System.out.println("Parent's display method");
    }
}

class Child extends Parent {
    @Override
    void display() {
        System.out.println("Child's display method");
    }
}

public class Main {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display(); // Calls the overridden method in Child
    }
}
```

Polymorphism (multiple behaviour)

Compile time polymorphism (Early Binding) - Behaviour (Which method will be executed) is decided at compile time based on parameters (Over loading)

Run time polymorphism (Late Binding) - Behaviour (Which method will be executed) is decided at run time (Over Riding)

Dynamic Method Dispatch:

Method overriding is one of the ways in which Java supports Runtime Polymorphism. Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.

When an overridden method is called through a superclass reference, Java determines which version(superclass/subclasses) of that method is to be executed based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.

At run-time, it depends on the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed

A superclass reference variable can refer to a subclass object. This is also known as upcasting. Java uses this fact to resolve calls to overridden methods at run time.

Syntax:

A obj = new B()

Reference Type: The reference type (A) determines which methods and fields can be accessed directly by the reference variable obj. Only the methods declared in class A are accessible via obj, even if B has additional methods.

Overridden Methods: If B overrides a method from A, calling that method on obj will execute B's version, not A's.

Abstract

- Abstract classes are used in situations in which we will want to define a superclass that declares the structure of a given abstraction without providing a complete implementation of every method. Sometimes we will want to create a superclass that only defines a generalization form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details
- We can not create object for abstract class
- We create object for class extending this class
- Class extending Abstract should implement the methods in abstract class for sure if it is not implementing this class should also be abstract class and class extending this class should implement that method
- We can have both concrete and abstract methods in abstract class
- Maintain mutable state

Interfaces (Generalization)

- Interfaces specify what a class must do and not how i.e. it just contain method declarations but not implementations.
- It is the blueprint of the class, just contain requirement specification through method declarations
- Interfaces allow multiple inheritance
- Interfaces cannot be instantiated
- Interfaces wont allow constructors
- Objects of the classes or interfaces implementing or extending them will be created
- Anonymous inner classes can also be used
- We cant declare instance and static blocks
- Serialization is achieved through marker interfaces
- Will not maintain state

All the methods in interface are by default public abstract, some

Features added:

Default Methods (Java 8): Allows interfaces to have methods with a body, providing default implementations that can be overridden by implementing classes.

->Default methods were introduced to provide code sharing.

Static Methods (Java 8): Enables utility methods within interfaces that can be called on the interface itself.

->Static Methods in Interface are those methods, which are defined in the interface with the keyword static. Unlike other methods in Interface, these static methods contain the complete definition of the function and since the definition is complete and the method is static, therefore these methods cannot be overridden or changed in the implementation class

Private Methods (Java 9): Facilitates code reuse within interfaces by allowing methods that are accessible only within the interface.

->to write sensitive code which you would not want to be accessed by any class or interface.

Private Static Methods (Java 9): Similar to private methods but applicable to static contexts within the interface.

All the variables in interface are by default public, final and static, and only those are possible

Differences between abstract class and interface:

1. An abstract class allows you to create functionality that subclasses can implement or override (partial abstraction). An interface only allows you to define functionality, not implement it (maximum abstraction) since Java 8 interfaces are also allowing concrete methods (default and static) (private from Java 9)
2. -An abstract class can have a constructor where as an interface cannot
3. Abstract classes can maintain mutable state (e.g., instance variables) because they can have fields that can be modified. Interfaces cannot have mutable state because any fields defined in an interface are implicitly public, static, and final, meaning they are constants and cannot be changed.
4. An abstract class can have static and instance blocks where as an interface cannot

In abstract class no limitations to modifiers for methods and variables

where as in Interfaces variables can only be public static final

methods can have abstract methods,

can have default and static methods (introduced in Java 8). (These are public implicitly)

can have private methods with the implementation (introduced in Java 9).

By default every method is public abstract (no need to mention)

-Abstract classes don't allow multiple inheritance where as interfaces allow

Types of Interfaces:

- Normal

Functional / SAM (Single Abstract Method)

Marker (No methods) - Blank interface to give permission for serialization and deserialization

Functional Interfaces

Functional Interfaces are interfaces which only contain single Abstract method

It can have other methods , but they should not be abstract

Lambda expressions

Lambda expression only work with functional interfaces

In lambda expressions variables need not contain data type before them in definition

In lambda expression if there is only single statement and it is return statement we should not mention return explicitly

If lambda expression contain single statement flower braces are not needed

Syntax:

parameter -> expression

(parameter1, parameter2) -> expression (more than 1 param)

(parameter1, parameter2) -> { code block } (expression more than single statement)

If the lambda expression needs to return a value, then the code block should have a return statement.

Syntax:

1. FuncInterface fobj = (int x)->System.out.println(2*x);

2 . ArrayList<Integer> nums = new ArrayList<Integer>();

nums. forEach(n -> {

if (n % 2 ==0) System.out.println(n); });

Anonymous Object

For the anonymous object, there is no reference.

Syntax:

```
new ClassName()
```

The only problem with this type of object is you can't reuse them

Inner Classes

->Class inside another class

inner class object instantiation syntax looks like

- OC.IC ICobj = OCobj.new IC(); (because InnerClass is non static)

If inner class is Static , then

- OC.IC ICobj = new OC.IC();

static can be only used for inner classes

Local Inner Class (defined inside a method): Cannot be public, protected, or private. It is implicitly package-private.

Anonymous inner class

Innerclasses without name used to implement abstract classes and interfaces without creating a separate class

Syntax:

```
1.MyInterface obj = new MyInterface() {
```

```
public void display() {
```

```
System.out.println("Anonymous inner class implementation");
```

```
};
```

```
2.MyInterface obj = () -> System.out.println("Lambda expression implementation");
```

Upcasting:

Upcasting is the typecasting of a child object to a parent object. Upcasting can be done implicitly. Upcasting gives us the flexibility to access the parent class members but it is not possible to access all the child class members using this feature. Instead of all the members, we can access some specified members of the child class. For instance, we can access the overridden methods.

```
Parent parent = new Child(); // Upcasting
```

Downcasting:

Similarly, downcasting means the typecasting of a parent object to a child object. Downcasting cannot be implicit.

```
Child child = (Child) parent; // Downcasting
```

Note: Downcasting should be done with caution. It's often a good idea to use instanceof to check the actual type before downcasting:

```
if (parent instanceof Child) {
    Child child = (Child) parent; // Safe downcasting
}
```

Enum(enumeration)

- Named constants
- An enum is a special "class" that represents a group of constants (public, static and final (unchangeable - cannot be overridden))
- Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.
- Every Enum we create is a class
- Enum cannot be used to create objects
- Constructors in Enum should only be private

But the only thing is an Enum cant be extended and cant extend also because

1.Compiler turns the enum into a subclass of the abstract class java.lang.Enum

2.Compiler compiles the enum as a final class

An Enum can be inside a class

ex: enum Status

```
{
    Running,Failed,Success
}
```

In Main class:

```
Status s = Status.Running;
```

```
print s -> Running
```

```
Now, print s.ordinal() - 0 (1st element)
```

```
Status[] ss = Status.values();
```

```
print ss -> some hexadecimal value
```

```
print ss[index] - that particular value will be printed
```

Lets see Enum with switch and if

```
Status s=Status.Pending;
```

Switch with Enum:

```
switch(s)
{
    case Running:
        System.out.println("All Good");
        Break;
    case Failed:
        System.out.println("Try Again");
        Break;
    case Pending:
        System.out.println("Please Wait");
        break;
    default:
        System.out.println("Done");
        break;
}
```

If with Enum: if(s==Status.Running)

```
System.out.println("All Good");
```

Enum with constructor and methods look like:

```
enum Laptop{  
    //    Mackbook(2000), XPS(2200), Surface(1500), ThinkPad(1800);  
    Mackbook(2000), XPS(2200), Surface, ThinkPad(1800);  
  
    private int price;  
  
    private Laptop()  
    {  
        price=500;  
    }  
  
    private Laptop(int price)  
    {  
        this.price=price;  
    }  
  
    public int getPrice()  
    {  
        return price;  
    }  
  
    public void setPrice(int price)  
    {  
        this.price=price;  
        System.out.println("in Laptop" + this.name());  
    }  
}
```


Annotations:

- Annotations start with '@'.
- Annotations do not change the action of a compiled program.
- Annotations help to associate metadata (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
- Annotations are not pure comments as they can change the way a program is treated by the compiler. See below code for example.
- Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.
- We can also specify retention(range - up to compile time or run time for custom made Annotations

Threads:

- A thread is a lightweight sub-process, the smallest unit of processing. Threads allow multiple tasks to run concurrently within a program.
- Parallelism: Achieved on multi-core processors; threads truly run at the same moment.
- Concurrency: Achieved via time-sharing; threads take turns running on the CPU, appearing to work simultaneously.
- Threads can be created using Class extending Thread and also by implementing runnable interface

->If we created using extending Thread

obj.start() starts the thread that is run the run() method, where obj is object of class extending Thread

->If we created using implementing Runnable, pass object of class to Thread object

Thread t1 = new Thread(obj);

t1.start() starts the thread that is run the run() method

Runnable is a functional interface , we can use lambda functions and need not create separate classes(using anonymous classes)

ex:

Runnable obj1=()->

```
{
    // write function logic
};
```

Thread states:

new Thread()- new <Creating Thread >

start - Runnable <executing and waiting for scheduler>

run - runnable <running on cpu>

sleep - waiting<keep thread in hold>

wait - waiting & notify - take back to runnable state

stop - goes to dead state <can go from runnable and running> - This is deprecated don't use (causes instability in s/w) Thread automatically goes to dead state when work is finished

Thread Priority:

Thread priority can be set using Thread.setPriority(int), suggesting the scheduler to give preferential execution time.

The actual execution order is ultimately decided by the JVM scheduler.

Race Condition:

A race condition occurs when multiple threads access shared data concurrently and try to modify it at the same time.

This can be avoided by making critical sections of code thread-safe using synchronized blocks, ReentrantLock, or other concurrency mechanisms.

Wrapper Classes

A Wrapper class in Java is a class whose object wraps or contains primitive data types.

We can store primitive values in wrapper class

Storing a primitive value to Wrapper class is boxing

When Assigning a primitive value to Wrapper class boxing will be automatic - Auto-boxing

Taking primitive value from Wrapper class is unboxing - ex: `int num = num1.intValue()` where `num1` is of type `Integer`

When taking a primitive value from Wrapper class boxing can be automatic - Auto-unboxing

Need of Wrapper Classes:

They convert primitive data types into objects. Objects are needed if we wish to modify the arguments passed into a method (because primitive types are passed by value).

The classes in `java.util` package handles only objects and hence wrapper classes help in this case also.

Data structures in the Collection framework, such as `ArrayList` and `Vector`, store only objects (reference types) and not primitive types.

An object is needed to support synchronization in multithreading.

Advantages of Wrapper Classes:

Collections allowed only object data.

On object data we can call multiple methods `compareTo()`, `equals()`, `toString()`

Cloning process only objects

Object data allowed null values.

Serialization can allow only object data

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Generic classes

Generic means parameterized types. Using generics, the idea is to allow any data type to be it Integer, String, or any user-defined Datatype and it is possible to create classes that work with different data types.

A Generic class simply means that the items or functions in that class can be generalized with the parameter(example T) to specify that we can add any type as a parameter in place of T like Integer, Character, String, Double or any other user-defined type.

- Type safety
- Compile time checking
- No type casting

Syntaxes(different cases):

```
public class GD<T>
```

In GD class outside main

```
T data[] = (T[])new Object()
```

Inside main

```
GD<String> gd = new GD()
```

If i dont specify param such as String any type will be accepted (considered as Object type)

->We can pass 2 param also<Shown below>

```
public class GD<K,T>
```

```
GD<String,Integer> gd = new GD()
```

->All methods of GD can be done in GD2 but only using Strings <Shown below>

```
public class GD<T>
```

In GD class outside main

```
T data[] = (T[])new Object()
```

Inside main

```
GD<String> gd = new GD()
```

```
public class GD2 extends GD<String>
```

```
public class GD2<T> extends GD<T>
```

If param is not mentioned for parent we should make child also generic

```
public class GD<T extends Number>
```

Only those who are inhereting from number

```
public class A
```

```
public class B extends A
```

```
public class GD<T extends A>
```

Can be done for user defined classes also

Now A type and B type are acceptable

```
public interface A
```

```
public class GD<T extends A>
```

extends only not implements

Generic methods

```
public void <T> show(E[] list) //E... list
```

```
{
```

```
    for (E c:list)
```

```
        Sop(x);
```

```
}
```

```
public void <T extends Number> show(E[] list)
```

```
{
```

```
    for (E c:list)
```

```
        Sop(x);
```

```
}
```

Java provides two interfaces to sort objects using data members of the class:

1.Comparable

2.Comparator

Comparable:

A comparable object is capable of comparing itself with another object. The class itself must implements the `java.lang.Comparable` interface to compare its instances.

It is strongly recommended (though not required) that natural orderings be consistent with equals. This is so because sorted sets (and sorted maps) without explicit comparators behave "strangely" when they are used with elements (or keys) whose natural ordering is inconsistent with equals. In particular, such a sorted set (or sorted map) violates the general contract for set (or map), which is defined in terms of the equals method.

For example, if one adds two keys `a` and `b` such that `(!a.equals(b) && a.compareTo(b) == 0)` to a sorted set that does not use an explicit comparator, the second add operation returns false (and the size of the sorted set does not increase) because `a` and `b` are equivalent from the sorted set's perspective.

Solution: Ensure `compareTo` and `equals` use the same criteria, or provide a custom `Comparator` to align their logic, preventing inconsistencies in comparisons and equality checks.

Virtually all Java core classes that implement `Comparable` have natural orderings that are consistent with equals. One exception is `java.math.BigDecimal`, whose natural ordering equates `BigDecimal` objects with equal values and different precisions (such as `4.0` and `4.00`).

Comparator:

A comparison function, which imposes a total ordering on some collection of objects. Comparators can be passed to a sort method (such as `Collections.sort` or `Arrays.sort`) to allow precise control over the sort order. Comparators can also be used to control the order of certain data structures (such as sorted sets or sorted maps), or to provide an ordering for collections of objects that don't have a natural ordering.

Unlike `Comparable`, `Comparator` is external to the element type we are comparing. It's a separate class. We create multiple separate classes (that implement `Comparator`) to compare by different members.

`Collections` class has a second `sort()` method and it takes `Comparator`. The `sort()` method invokes the `compare()` to sort objects.

To use comparator:

Create a class that implements Comparator (and thus the compare() method that does the work previously done by compareTo()).

Make an instance of the Comparator class.

Call the overloaded sort() method, giving it both the list and the instance of the class that implements Comparator.

If a Comparator is inconsistent with equals, Collections like TreeSet and TreeMap depends heavily on the consistency between the ordering imposed by compareTo or Comparator and the equals method. Ensuring consistency between these methods avoids unexpected behavior, such as duplicates being allowed in a set.

Comparators should ideally implement java.io.Serializable because they might be used in serializable data structures (like TreeSet or TreeMap). If a Comparator is part of a data structure that needs to be serialized, the serialization will only work if the Comparator itself is serializable.

Comparable Vs Comparator:

- Comparable is meant for objects with natural ordering which means the object itself must know how it is to be ordered. For example Roll Numbers of students. Whereas, Comparator interface sorting is done through a separate class.
- Natural ordering refers to the default sorting order that objects follow when they implement the Comparable interface in Java.
ex: For numbers, the natural ordering is ascending order (e.g., 1, 2, 3).
For strings, the natural ordering is lexicographical (dictionary-like) order (e.g., "apple", "banana", "cherry").
For dates, the natural ordering is chronological order (earliest to latest).
- Logically, Comparable interface compares “this” reference with the object specified and Comparator in Java compares two different class objects provided.
- If any class implements Comparable interface in Java then collection of that object either List or Array can be sorted automatically by using Collections.sort() or Arrays.sort() method and objects will be sorted based on there natural order defined by CompareTo method.
- A basic differentiating feature is that using comparable we can use only one comparison. Whereas, we can write more than one custom comparators as you want for a given type, all using different interpretations of what sorting means. Like in the comparable example we could just sort by only one attribute, i.e., year but in the comparator, we were able to use different attributes

Comparable: A class implementing Comparable does not inherently support comparing null values. Attempting to use `compareTo` with null will lead to exceptions if not handled explicitly, you have to handle nulls explicitly within the class itself, mixing ordering logic with class implementation, leading to a single, inflexible ordering approach.

Comparator: Provides more flexibility for handling null values. You can design a comparator to handle null arguments in a way that suits your application's needs, such as treating null as less than or greater than any non-null value. This is useful for sorting collections where some elements might be null. , allows you to externalize and customize null handling and ordering logic in various ways, making it much cleaner and easier to manage.

To summarize, if sorting of objects needs to be based on natural order then use Comparable whereas if you sorting needs to be done on attributes of different objects, then use Comparator in Java.

Collections

Array:

Fixed size; cannot be modified.

Requires creating a new array to change size.

Collection (I)(java.util package):

Collection extends iterable

Variable-sized collections: ArrayList, LinkedList.

Distinct collections: Set.

Sorted collections: SortedSet.

Collections are used to group elements.

List:

Ordered group of elements with indices.

Can contain duplicate values.

Implementations: ArrayList, LinkedList, Vector.

Legacy class: Stack (extends Vector).

Set:

Unordered group of elements, cannot have duplicates.

Preferred for frequent search requirements.

Implementations:

HashSet and LinkedHashSet (use hash codes, unordered).

SortedSet (extends Set).

TreeSet (implements SortedSet, uses a Red-Black Tree, sorted order).

Queue:

Follows FIFO (First In, First Out) principle.

PriorityQueue implements Queue; by default, it acts as a min-heap.

Deque (Double-Ended Queue) extends Queue, allowing insertion and deletion from both ends.

ArrayDeque implements Deque; it provides constant time operations and is more efficient than Stack and LinkedList for stack and queue operations.

Map:

Represents key-value pairs.

Implementations:

SortedMap (extends Map).

TreeMap (implements SortedMap, uses a Red-Black Tree; maintains sorted order).

HashMap, LinkedHashMap, Hashtable (legacy class).

LinkedHashMap maintains insertion order using linked lists; can also maintain access order when specified.

ArrayList vs. LinkedList:

ArrayList has better random access performance.

LinkedList is better for frequent insertions/deletions as it doesn't require shifting elements and uses more space for node links (java uses doubly linked list).

LinkedList extends Deque

Set Implementations:

TreeSet uses a tree structure; operations take $O(\log n)$ time and elements are sorted based on natural ordering or a comparator.

HashSet uses hashing for quick operations (add, remove, contains) in constant time; unordered.

Linked HashSet-insertion order is followed, duplicates not allowed, LinkedHashMap provides constant-time performance for basic operations such as add, remove, and contains.

Deque:

ArrayDeque is stored internally as an array and provides constant time operations.

It is preferred over Stack for stack operations and LinkedList for queue operations due to performance benefits.

PriorityQueue:

Array-Based Implementation

Represents a heap (min-heap by default); elements with the lowest values have the highest priority.

Operations are $O(\log n)$ due to the binary tree structure.

Behavior can be customized using a comparator.

Hashing:

Uses hash functions to determine storage location.

Open Addressing: Handles collisions by placing elements in the next available spot.

Chaining: Uses linked lists to handle collisions at the same index.

Hash table's starting size is 16, with a typical load factor of 0.75, optimizing time-space trade-offs.

Load Factor: It is a measure that determines when to resize the hash table (rehash). It is defined as:

Load Factor = Number of Elements / Capacity of the Hash Table

Typical Value: The default load factor is 0.75. This means the hash table will resize when it is 75% full.

Map Implementations:

TreeMap: Provides sorted key-value pairs using a Red-Black tree, with operations taking $O(\log n)$ time.

HashMap: Provides constant time operations; does not maintain order.

LinkedHashMap: Maintains insertion or access order; can limit size by overriding `removeEldestEntry()`. (used for cache)

HashMap is generally faster for iteration due to its simpler internal structure.

LinkedHashMap may be slightly slower because of the overhead of maintaining the linked list for ordering.

Legacy Classes: Stack, Vector and Hashtable are considered legacy classes; avoid using them in favor of modern collections like ArrayDeque and HashMap.

Hashtable: key-value, it treats key and value as objects since this doesn't support generics

Properties: Designed for string key-value pairs, despite inheriting from `Hashtable<Object, Object>`. Use only strings to ensure proper functionality.

Purpose: Stores configuration data as key-value pairs

Can save and load key-value pairs from text files and xml files

StringTokenizer: Used for splitting strings in a single String based on delimiters.

BitSet: An array of bits used to perform bitwise operations, useful for finding duplicates, unions, or missing elements.

Note: We can create object outside methods inside a class for making instances of our class to maintain the state of another object

Ex : `List<String> list = new ArrayList<>();` , `ArrayList<Student> al = new ArrayList<Student>();`

In general Classes directly only contains fields and assignments(if done), they don't contain other executable statements directly in it, they should be there in blocks, methods or constructors. Executable statements, like function calls or loops, are not allowed directly inside the class body outside of methods, constructors, or static/instance initialization blocks.

final

The final keyword in Java is used as a non-access modifier applicable only to a variable, a method, or a class.

Final variables: When a variable is declared as final, its value cannot be changed once it has been initialized. This is useful for declaring constants or other values that should not be modified.

Final methods: When a method is declared as final, it cannot be overridden by a subclass. This is useful for methods that are part of a class's public API and should not be modified by subclasses.

Final classes: When a class is declared as final, it cannot be extended by a subclass. This is useful for classes that are intended to be used as is and should not be modified or extended.

Initialization: Final variables must be initialized either at the time of declaration or in the constructor of the class. This ensures that the value of the variable is set and cannot be changed.

Performance: The use of a final can sometimes improve performance, as the compiler can optimize the code more effectively when it knows that a variable or method cannot be changed.

Security: The final can help improve security by preventing malicious code from modifying sensitive data or behavior.

If the final variable is a reference, this means that the variable cannot be re-bound to reference another object, but the internal state of the object pointed by that reference variable can be changed i.e. you can add or remove elements from the final array or final collection.

It is good practice to represent final variables in all uppercase, using underscore to separate words.

Stream API

The key abstraction introduced in this package is stream. The classes Stream, IntStream, LongStream, and DoubleStream are streams over objects and the primitive int, long and double types.

Streams differ from collections in several ways:

- No storage. A stream is not a data structure that stores elements; instead, it conveys elements from a source such as a data structure, an array, a generator function, or an I/O channel, through a pipeline of computational operations.
- Functional in nature. An operation on a stream produces a result, but does not modify its source. For example, filtering a Stream obtained from a collection produces a new Stream without the filtered elements, rather than removing elements from the source collection.

- Laziness-seeking. Many stream operations, such as filtering, mapping, or duplicate removal, can be implemented lazily, exposing opportunities for optimization. For example, "find the first String with three consecutive vowels" need not examine all the input strings. Stream operations are divided into intermediate (Stream-producing) operations and terminal (value- or side-effect-producing) operations. Intermediate operations are always lazy.
- Possibly unbounded. While collections have a finite size, streams need not. Short-circuiting operations such as `limit(n)` or `findFirst()` can allow computations on infinite streams to complete in finite time.
- Consumable. The elements of a stream are only visited once during the life of a stream. Like an Iterator, a new stream must be generated to revisit the same elements of the source.

Streams can be obtained in a number of ways. Some examples include:

- From a Collection via the `stream()` and `parallelStream()` methods;
- From an array via `Arrays.stream(Object[])`;
- From static factory methods on the stream classes, such as `Stream.of(Object[])`, `IntStream.range(int, int)` or `Stream.iterate(Object, UnaryOperator)`;
- The lines of a file can be obtained from `BufferedReader.lines()`;
- Streams of file paths can be obtained from methods in `Files`;
- Streams of random numbers can be obtained from `Random.ints()`;
- Numerous other stream-bearing methods in the JDK, including `BitSet.stream()`, `Pattern.splitAsStream(java.lang.CharSequence)`, and `JarFile.stream()`

Benefits of Java Stream:

- No Storage
- Pipeline of Functions
- Laziness
- Can be infinite
- Can be parallelized
- Can be created from collections, arrays, Files Lines, Methods in Stream, IntStream etc.

There are two types of Operations in Streams:

1. Intermediate Operations
2. Terminate Operations

Characteristics of Intermediate Operations:

- Methods are chained together.
- Intermediate operations transform a stream into another stream.
- It enables the concept of filtering where one method filters data and passes it to another method after processing.

Methods:

- flatMap(List::stream): Flattens the nested lists into a single stream of strings.
- filter(s -> s.startsWith("S")): Filters the strings to only include those that start with "S".
- map(String::toUpperCase): Converts each string in the stream to uppercase.
- distinct(): Removes any duplicate strings.
- sorted(): Sorts the resulting strings alphabetically.
- peek(...): Adds each processed element to the intermediateResults set for intermediate inspection.
- collect(Collectors.toList()): Collects the final processed strings into a list called result.

Terminal Operations

- Terminal Operations are the type of Operations that return the result. These Operations are not processed further just return a final result value.

Methods:

- forEach: Prints each name in the list.
- collect: Filters names starting with 'S' and collects them into a new list.
- reduce: Concatenates all names into a single string.
- count: Counts the total number of names.
- findFirst: Finds and prints the first name in the list.
- allMatch: Checks if all names start with 'S'.
- anyMatch: Checks if any name starts with 'S'.

Example:

```
import java.util.Arrays;

import java.util.HashSet;

import java.util.List;

import java.util.Set;

import java.util.stream.Collectors;

public class StreamIntermediateOperationsExample {

    public static void main(String[] args) {
```



```

//list of list of names
List<List<String>> listOfLists = Arrays.asList(
    Arrays.asList("Reflection", "Collection", "Stream"),
    Arrays.asList("Structure", "State", "Flow"),
    Arrays.asList("Sorting", "Mapping", "Reduction", "Stream")
);

// Create a set to hold intermediate results
Set<String> intermediateResults = new HashSet<>();

// Stream pipeline demonstrating various intermediate operations
List<String> result = listOfLists.stream()
    .flatMap(List::stream)      // Flatten the list of lists into a single stream
    .filter(s -> s.startsWith("S")) // Filter elements starting with "S"
    .map(String::toUpperCase)   // Transform each element to uppercase
    .distinct()                 // Remove duplicate elements
    .sorted()                   // Sort elements
    .peek(s -> intermediateResults.add(s)) // Perform an action (add to set) on each element
    .collect(Collectors.toList()); // Collect the final result into a list

// Print the intermediate results
System.out.println("Intermediate Results:");
intermediateResults.forEach(System.out::println);

// Print the final result
System.out.println("Final Result:");
result.forEach(System.out::println);
}
}

```

Parallel Streams:

Java Parallel Streams is a feature of Java 8 and higher, meant for utilizing multiple cores of the processor. Normally any java code has one stream of processing, where it is executed sequentially. Whereas by using parallel streams, we can divide the code into multiple streams that are executed in parallel on separate cores and the final result is the combination of the individual outcomes. The order of execution, however, is not under our control.

Therefore, it is advisable to use parallel streams in cases where no matter what is the order of execution, the result is unaffected and the state of one element does not affect the other as well as the source of the data also remains unaffected.

There are two ways we can create

1. Using `parallel()` method on a stream

Syntax: `sobj.parallel().forEach(System.out::println);`

Where `sobj` is stream object

2. Using `parallelStream()` on a Collection

Syntax: `cobj.parallelStream().forEach(System.out::println);`

Where `cobj` is object of a collection or its child or theirs..

Optional Class:

A container object which may or may not contain a non-null value. If a value is present, `isPresent()` will return true and `get()` will return the value.

Additional methods that depend on the presence or absence of a contained value are provided, such as [orElse\(\)](#) (return a default value if value not present) and [ifPresent\(\)](#) (execute a block of code if the value is present).

This is a [value-based](#) class; use of identity-sensitive operations (including reference equality (`==`), identity hash code, or synchronization) on instances of `Optional` may have unpredictable results and should be avoided.

Ex:

```
Optional<String> checkNull = Optional.ofNullable(words[5]);
```

```
    if (checkNull.isPresent())
```

```
        System.out.print(word);
```

```
    else
```

```
        System.out.println("word is null");
```

Method Reference:

You use [lambda expressions](#) to create anonymous methods. Sometimes, however, a lambda expression does nothing but call an existing method. In those cases, it's often clearer to refer to the existing method by name. Method references enable you to do this; they are compact, easy-to-read lambda expressions for methods that already have a name.

Ex:

```
Arrays.sort(rosterAsArray, Person::compareByAge);
```

The method reference `Person::compareByAge` is semantically the same as the lambda expression `(a, b) -> Person.compareByAge(a, b)`. Each has the following characteristics:

- Its formal parameter list is copied from `Comparator<Person>.compare`, which is `(Person, Person)`.
- Its body calls the method `Person.compareByAge`.

Kinds of Method References

There are four kinds of method references:

Kind	Syntax	Examples
Reference to a static method	<code>ContainingClass::staticMethodName</code>	<code>Person::compareByAge</code> <code>MethodReferencesExamples::appendStrings</code>
Reference to an instance method of a particular object	<code>containingObject::instanceMethodName</code>	<code>myComparisonProvider::compareByName</code> <code>myApp::appendStrings2</code>
Reference to an instance method of an arbitrary object of a particular type	<code>ContainingType::methodName</code>	<code>String::compareToIgnoreCase</code> <code>String::concat</code>
Reference to a constructor	<code>ClassName::new</code>	<code>HashSet::new</code>

Reference to a constructor is known as Constructor reference

This is a short notes , there are many topics and methods (All may not be mentioned even in mentioned topics) refer to them in Java Doc (Documentation) or any popular online sources.

-By Abhijith